

1. Introduction

1.1 Introduction

Governance forms the backbone of national development, and in India, rural governance holds a particularly vital role. The Gram Panchayat serves as the first point of interaction between the government and rural citizens. However, the current system largely depends on outdated, manual processes for managing records, resolving grievances, collecting taxes, and implementing welfare schemes. These manual methods are time-consuming, prone to errors, and lack transparency, which leads to inefficiency and frustration among citizens.

To address these challenges, the AI Powered Rural Governance Support System (AIRGSS) aims to modernize and streamline Panchayat-level governance by introducing automation, transparency, and inclusivity. The proposed system will integrate Artificial Intelligence (AI), Natural Language Processing (NLP), and multilingual voice/chat interfaces to make government services more accessible and efficient.

This SRS document outlines the detailed requirements, objectives, functionalities, and design considerations of the system to ensure proper development, testing, and deployment.

1.2 Problem Description

Rural administrative systems face several persistent challenges, including:

- Manual and paper-based processes for grievance handling, taxation, and fund management.
- Lack of transparency in fund utilization and project execution.
- Limited awareness of government schemes and their eligibility criteria.
- Low digital literacy and language barriers preventing citizens from using online platforms.

These issues result in delays, inefficiency, and a lack of trust in governance systems. A comprehensive, intelligent, and accessible solution is required to enhance citizen engagement and administrative efficiency.

1.2 Problem Statement

People in rural areas find it difficult to access government services due to manual operations, lack of awareness, and low transparency. A single, AI-driven platform is needed to connect citizens and administrators, enabling efficient communication, faster grievance resolution, and transparent governance.

1.3 Objectives

The main objectives of this system are:

1. To build an AI-powered grievance management system that allows citizens to register complaints via text or voice, automatically categorize them using NLP, assign them to the appropriate department, and track their status in real time.
2. To enhance the performance, transparency, and accessibility of governance using multilingual chatbot support, voice-enabled interaction, and real-time data analytics.
3. To develop a comprehensive digital governance platform that integrates essential services such as tax and bill payments, fund tracking, digital record management, and AI-based government scheme recommendations.

1.4 Scope

a) Identification of the Software Product:

The software product to be developed is **AI Powered Rural Governance Support System (AIRGSS)** a web-based platform integrated with Artificial Intelligence, Natural Language Processing, and voice-enabled features to support efficient rural governance

b) Functionality of the Product:

The system will allow citizens to file grievances, pay taxes and bills, access welfare scheme information, and interact with officials through text or voice.

Administrators will manage funds, monitor projects, and maintain digital records through an integrated dashboard.

The system will not handle offline administrative operations or state-level decision-making beyond Panchayat jurisdiction.

c) Application, Benefits, and Goals:

The application will be used by rural citizens and Panchayat officials to improve communication, efficiency, and transparency. It aims to:

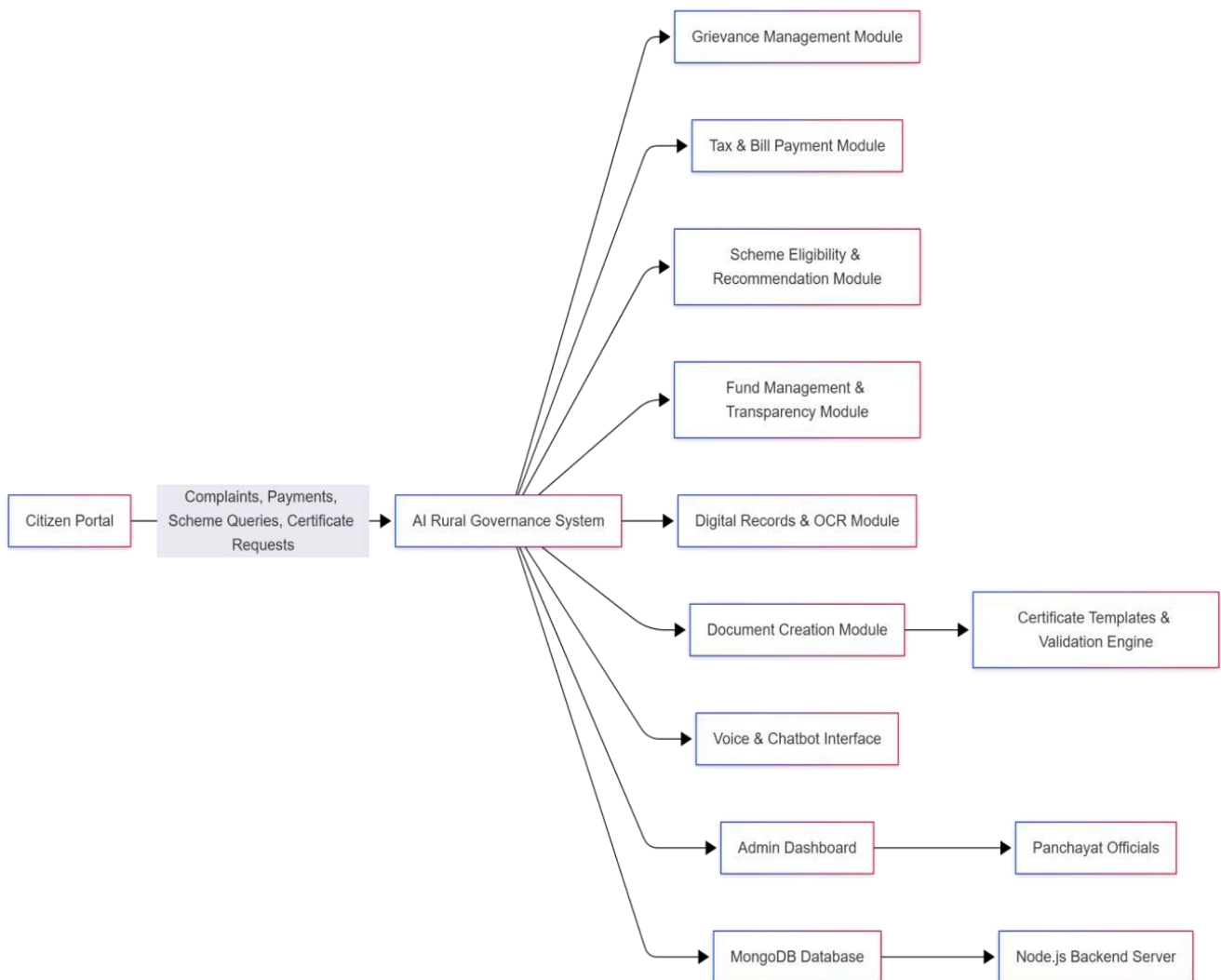
- Automate grievance and record management ● Provide real-time fund and project tracking.
- Ensure easy access to services through multilingual and voice support.
- Promote inclusive, transparent, and data-driven rural governance.

2. Overall description

2.1 Product perspective

The AI Powered Rural Governance Support System (AIRGSS) is a web-based e-Governance platform designed to streamline rural administrative functions. It integrates multiple services such as grievance management, tax and bill payments, fund monitoring, scheme recommendations, and digital record keeping into a single system. The platform uses AI and NLP technologies to automate complaint classification, predict scheme eligibility, and provide multilingual voice or chatbot support. It operates on a three-tier architecture: a React.js-based frontend, Node.js and Express.js backend, and a MongoDB database. The system will be accessible through standard web browsers and hosted on a cloud or local server. By connecting citizens and Panchayat administrators digitally, the system replaces manual processes with a transparent, efficient, and data-driven governance model.

2.1.1 Block Diagram



(2.1.1 BLOCK DIAGRAM)

The **AI Rural Governance System** provides digital governance services to citizens through a **Citizen Portal** for complaints, payments, scheme queries, and certificate requests.

It includes several modules **Grievance Management, Tax & Bill Payment, Scheme Eligibility & Recommendation**, and **Fund Management & Transparency** for efficient operations.

The **Digital Records & OCR Module** digitizes documents, while the **Document Creation Module** works with the **Certificate Templates & Validation Engine** to generate verified certificates.

A **Voice & Chatbot Interface** offers AI-based support to citizens in local languages.

The **Admin Dashboard** helps **Panchayat Officials** monitor and manage all activities.

Data is stored in the **MongoDB Database**, and system operations are handled by the **Node.js Backend Server**, ensuring smooth and secure performance.

2.1.2 Hardware Requirements

- Processor: Minimum Dual-Core CPU; Recommended Intel i5 or above.
- RAM: Minimum 4 GB; Recommended 8 GB for better performance.
- Storage: At least 1 GB of free disk space for files and database.
- Internet Connection: Minimum 1 Mbps for accessing APIs and hosting.
- Server Hardware (for deployment): Minimum 2 vCPUs, 8 GB RAM, and 50 GB storage (cloud or local server).

2.1.3 Software Requirements

- Operating System: Compatible with Windows, Linux, or macOS.
- Web Browser: Supports Google Chrome, Mozilla Firefox, Microsoft Edge, and Safari.
- Backend Environment: Node.js with Express.js framework.
- Frontend Technologies: HTML, CSS, JavaScript, and React.js.
- Database: MongoDB for storing and managing application data.
- Development Tools: Visual Studio Code
- Version Control: Git and GitHub for collaborative development.
- AI/NLP Libraries: TensorFlow, Hugging Face Transformers for intelligent grievance handling and chatbot features.

2.2 Product functions

The AI-Powered Rural Governance Support System is designed to simplify and automate various Panchayat-level administrative and citizen service processes. The system integrates multiple modules to provide transparency, efficiency, and accessibility in rural governance. The major functions of the system include:

1. Grievance Management

- Allows citizens to register complaints through text, web forms, or voice input.

- Automatically classifies and prioritizes grievances using Natural Language Processing (NLP)
- Generates a unique ticket ID and forwards it to the concerned department.
- Enables citizens and administrators to track complaint status in real time.

2. Tax & Bill Payment

- Facilitates secure online payments for property taxes, water, and utility bills through integrated payment gateways.
- Maintains detailed payment history and provides downloadable receipts.
- Sends automatic due reminders through SMS or WhatsApp.
- Provides officials with dashboards for tracking collections and revenue trends.

3. Scheme Eligibility & Recommendation

- Maintains a centralized database of welfare schemes with eligibility criteria.
- Uses AI/ML models to analyze citizen profiles and predict eligible schemes.
- Displays application steps and sends notifications for new or expiring schemes.

4. Fund Management & Transparency

- Tracks allocation, disbursement, and expenditure of Panchayat funds.
- Displays project progress and utilization details on an interactive dashboard.
- Allows citizens to view fund usage for improved transparency.

5. Digital Records & OCR

- Scans and digitizes legacy records using Optical Character Recognition (OCR).
- Provides searchable, role-based access to digital records for citizens and administrators.
- Maintains audit trails for all modifications and retrievals.

6. Document Creation & Validation

- Automates generation of official certificates using pre-defined templates
- Validates authenticity through digital signatures or QR codes.

7. Voice & Chatbot Assistance

- Provides AI-based chatbot for instant support in local languages.
- Supports speech-to-text for complaint registration and text-to-speech for updates.
- Works efficiently in low-bandwidth environments to ensure inclusivity.

8. Admin Dashboard

- Enables Panchayat officials to monitor complaints, payments, schemes, and fund usage.
- Offers analytics and performance reports for decision-making.

3. Specific requirements

This section describes in detail the functional and non-functional requirements of the **AI Powered Rural Governance Support System (AIPRGSS)**. It defines all inputs, outputs, and operations required for the system to function effectively, ensuring that designers can design and testers can verify the system against these specifications.

3.1 External interfaces

3.1.1 Inputs

- Citizen complaint or request submitted through:
 - Text form on the web portal.
 - Voice command through the chatbot interface.
 - Chatbot messages (in local language).
- Payment details for bills and taxes (via UPI, credit/debit card, or payment gateway).
- Citizen data for scheme eligibility (age, gender, income, occupation, family status).
- Uploaded scanned documents for OCR and record digitization.
- Administrative data entries such as fund allocations, expenditure, and project updates.
- Admin login credentials for system access and management.

3.1.2 Outputs

- Complaint acknowledgment and unique ticket number.
- Real-time complaint status updates.
- Digital receipts for bill and tax payments.
- AI-based scheme recommendations and eligibility results.
- Digitized records and downloadable verified documents (PDFs).
- Dashboard reports and analytics for administrators.
- Voice or text responses from the chatbot in the selected regional language.
- Public transparency dashboard showing fund utilization and ongoing projects.

3.2 Functional Requirements

The system shall provide the following functionalities to meet user and administrative needs.

3.2.1 Grievance Management

- The system **shall allow** citizens to submit grievances through text, voice, or chatbot.
- The system **shall automatically classify** complaints using Natural Language Processing (NLP).
- The system **shall generate** a unique ticket ID for every grievance.
- The system **shall route** the complaint to the concerned department.
- The system **shall provide** real-time complaint tracking to users.

3.2.2 Tax and Bill Payment

- The system **shall enable** secure online tax and bill payments through integrated payment gateways.
- The system **shall record** payment transactions in the database.
- The system **shall generate** a digital payment receipt.
- The system **shall send** due date reminders via SMS or WhatsApp.

3.2.3 Scheme Eligibility and Recommendation

- The system **shall maintain** a database of welfare schemes with eligibility criteria.
- The system **shall analyze** user profiles to predict suitable schemes using AI/ML models.
- The system **shall notify** users of new or expiring schemes.

3.2.4 Fund Management and Transparency

- The system **shall track** fund allocations, disbursements, and expenditures for Panchayat projects.
- The system **shall display** project progress and utilization on an interactive dashboard.
- The system **shall allow** citizens to view fund usage for transparency.

3.2.5 Digital Records and OCR

- The system **shall scan** and convert physical records into digital format using OCR.
- The system **shall make** digitized data searchable and accessible by role.
- The system **shall maintain** audit trails for every edit or access event.

3.2.6 Document Creation and Validation

- The system **shall automatically generate** official documents and certificates.
- The system **shall validate** authenticity using QR codes or digital signatures.

3.2.7 Voice and Chatbot Assistance

- The system **shall provide** a chatbot interface in regional languages.
- The system **shall convert** speech to text for complaint submission and text to speech for updates.
- The system **shall operate** efficiently in low bandwidth conditions.

3.2.8 Admin Dashboard

- The system **shall allow** officials to log in and monitor complaints, payments, and funds.
- The system **shall generate** analytical reports for performance monitoring.

3.3 Performance Requirements.

- The server shall have a **multi-core processor (minimum 8 cores, 3.0 GHz or higher)**.
- At least **32 GB RAM** shall be available for smooth multitasking and user handling.
- A minimum of **2 TB SSD storage** shall be provided for database and document storage.
- The system shall have a **high-speed network connection (1 Gbps or higher)** with backup power support.

3.4 Design constraints

1. Regulatory and Data-Privacy Compliance

- The system must follow **Government of India e-Governance and Data Privacy Standards**.
- All citizen information—complaints, payments, and documents—must be **encrypted** and accessible only to authorized users.
- Data storage and backup must comply with **legal retention and auditing rules**.

2. Hardware Limitations

- Minimum hardware: **4 GB RAM, 1 GB free disk space**, and a stable internet connection.
- OCR accuracy may vary with **scan quality**; low-resolution images can reduce recognition precision.
- System performance depends on **server capacity** and **network bandwidth** in rural areas.

3. Software and Technology Constraints

- Backend uses **Node.js + Express.js**, and frontend uses **React.js + TailwindCSS**; proper version compatibility is required.
- Database must run on **MongoDB (NoSQL)**.
- The application supports only modern browsers—**Chrome, Firefox, Edge, Safari**.
- An **active internet connection** is required for real-time functions, though some chatbot actions can queue offline.

4. Language and Accessibility Constraints

- Voice and chatbot modules depend on **regional-language accuracy**; accent or noise variations may affect performance.
- The user interface must remain **simple and accessible** to citizens with limited digital literacy.

3.5 Logical database requirements

The system requires a well-structured database to manage citizen information, administrative records, payments, and documents efficiently. The database will be implemented using **MongoDB (NoSQL)** to store both structured and unstructured data.

a) Types of Information Used by Various Functions

- **Citizen Information:** Personal details, contact number, address, login credentials.
- **Grievance Data:** Complaint ID, category, description, status, and resolution updates.
- **Payment Records:** Tax and bill details, transaction ID, payment date, amount, and receipt.
- **Scheme Data:** List of welfare schemes, eligibility criteria, and recommended beneficiaries.
- **Fund Management Data:** Allocation, disbursement, and project expenditure details.
- **Documents:** Scanned files, OCR text data, and generated certificates with validation codes.
- **Chatbot Logs:** User queries and corresponding AI-generated responses for analytics. **b)**

Frequency of Use

- **Citizen interactions:** Frequent (daily/weekly) for grievances, payments, and document requests.
- **Admin operations:** Regular (weekly/monthly) for fund management and report generation.
- **System backups:** Performed automatically **once per day**.
- **Data retrieval:** On-demand for all user queries and dashboard analytics.

c) Data Entities and Relationships

- **Entities:** Citizen, Complaint, Payment, Scheme, Fund, Document, Admin.
- **Relationships:**
 - One *Citizen* can have many *Complaints* and *Payments*.
 - One *Citizen* can be linked to multiple *Schemes*.
 - One *Admin* manages multiple *Funds* and *Documents*.
 - Each *Complaint* and *Transaction* has a unique identifier linked to a *Citizen*. **d)**

Integrity Constraints

- Each record (Citizen, Complaint, Payment) must have a **unique ID**.
- Foreign key references must maintain consistency between citizen and complaint/payment entities.
- Only **authenticated users** can create, update, or delete their own records.
- System must enforce **data validation rules** (e.g., numeric amount fields, valid email, and date formats).

- All data transactions must maintain **atomicity and consistency** even during power or network failures.

e) Data Retention Requirements

- Citizen and transaction data must be retained for a minimum of **5 years**.
- Log files and chatbot conversations should be stored for **6 months** for performance analytics.
- Backup copies of the database should be maintained **daily** and archived **monthly**.
- Deleted user records should be kept in an **inactive archive** for 1 year before permanent removal.

3.6 Software system attributes

3.6.1 Reliability

- The system shall maintain a **minimum reliability of 98%**, ensuring consistent performance under normal operating conditions.
- It shall include **automatic error detection and recovery mechanisms** to minimize downtime.
- All critical transactions such as complaint submission and payment processing shall be **validated and logged** to prevent data loss.
- Database redundancy through **MongoDB** shall ensure fault tolerance and data consistency.

3.6.2 Availability

- The system shall ensure **high availability (98% uptime)** for both citizens and administrators.
- **Cloud hosting and backup mechanisms** shall guarantee service continuity during hardware or network failures.
- **Checkpoint and recovery features** shall restore interrupted sessions or transactions.
- Routine **maintenance and updates** shall be scheduled to minimize operational disruption.

3.6.3 Security

- The system shall implement **secure login and authentication** for all users.
- All data transmission shall be **encrypted using SSL/TLS** to protect user information.
- Sensitive data (citizen records, payments, etc.) shall be **encrypted at rest and in transit**.
- **Role-based access control (RBAC)** shall restrict access to system modules based on user roles.
- Regular **security audits and vulnerability testing** shall be conducted to ensure protection from threats.

3.6.4 Maintainability

- The system shall follow a **modular design** to allow updates without affecting other components.
- **Code documentation and version control (Git)** shall be maintained for easy modification.
- Automated **error logging and monitoring tools** shall assist in identifying and resolving issues.
- The architecture shall allow **scalability and integration** of new features with minimal effort.

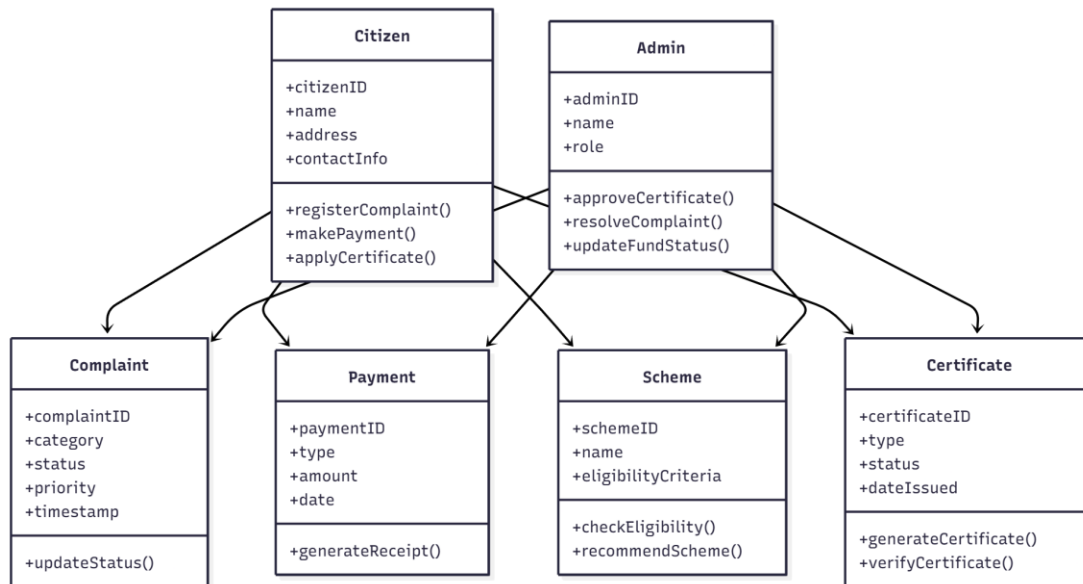
3.6.5 Portability

- The system shall be **platform-independent**, deployable on **Windows, Linux, or macOS**.
- It shall operate smoothly across major browsers such as **Chrome, Firefox, Edge, and Safari**.
- The system shall support **cloud and on-premise deployment environments**.
- Responsive design using **React.js** shall ensure compatibility across desktops and mobile devices.

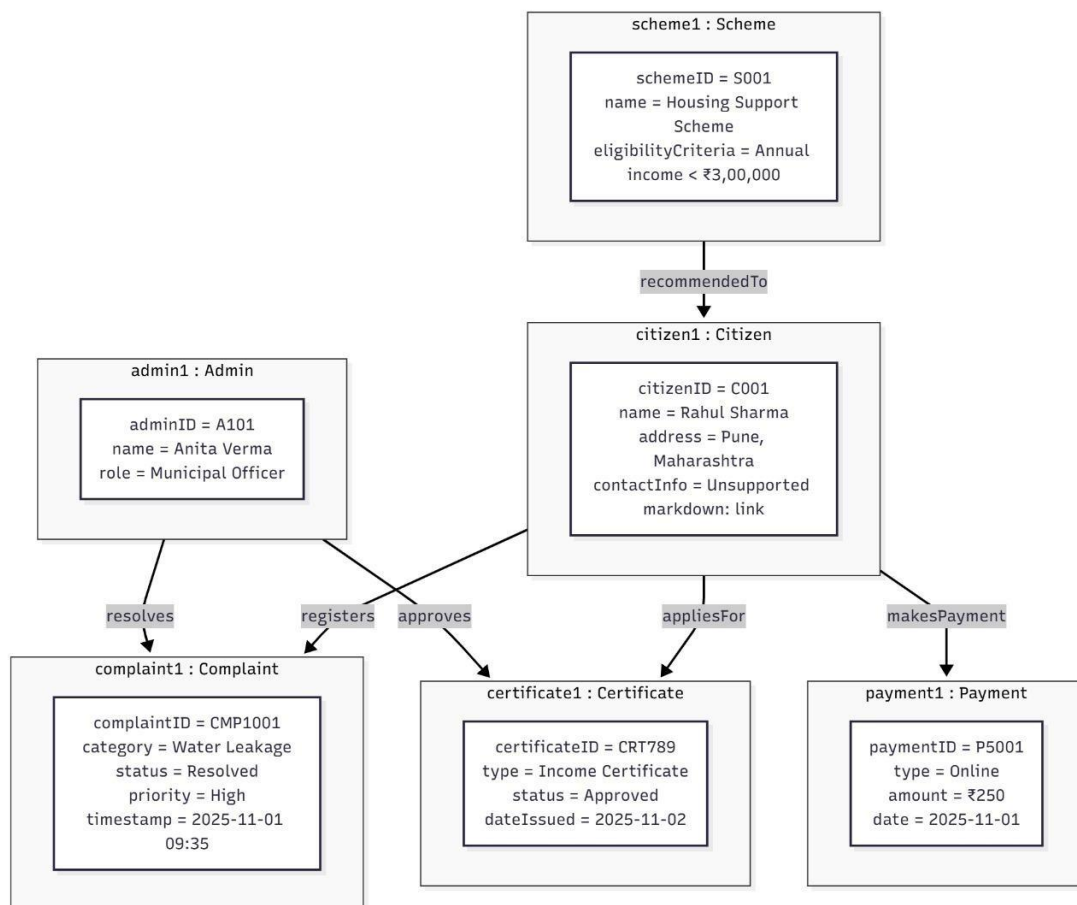
4. Software Design Document

4.1. Structural Design

a. Class Diagrams

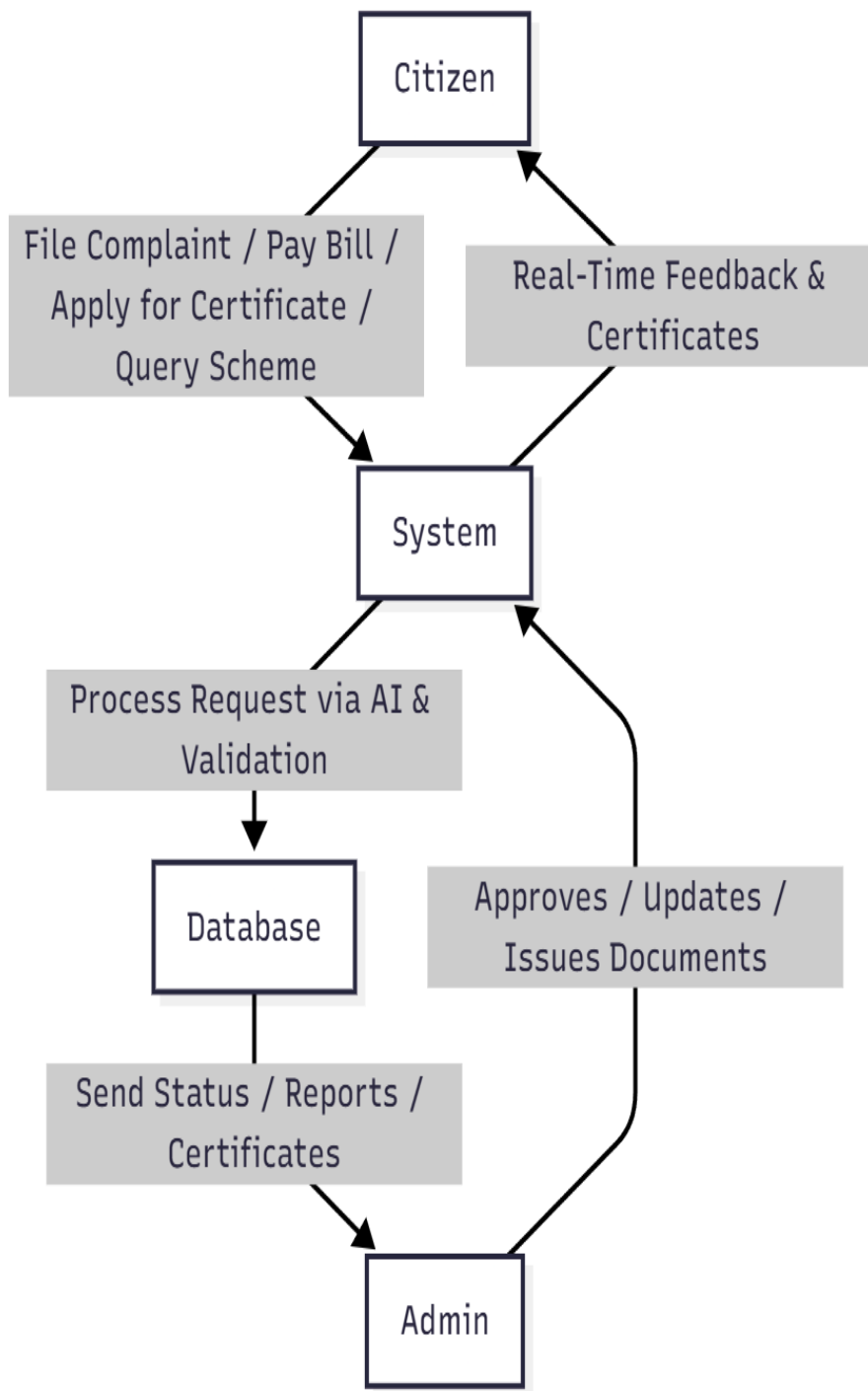


b. Object Diagrams

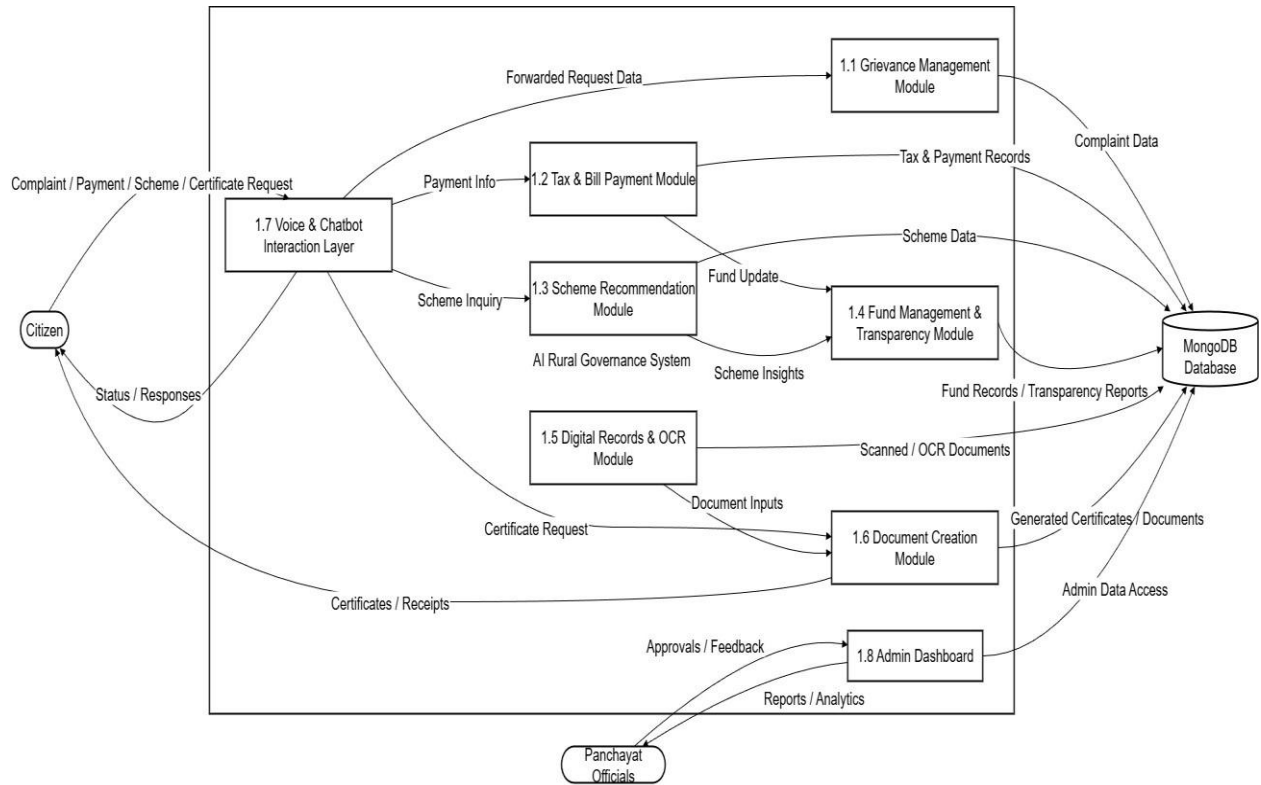


4.2. Data Design

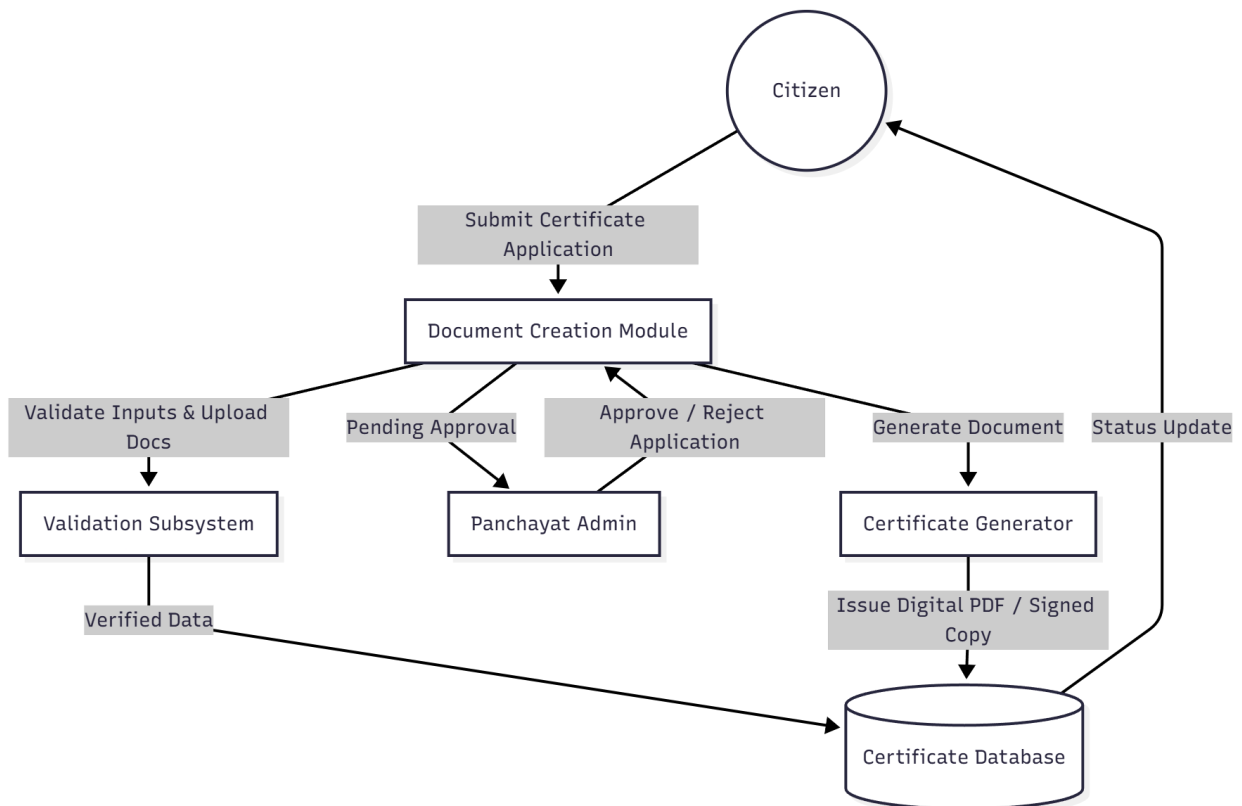
a. Data-flow Diagram



(Dfd 0)

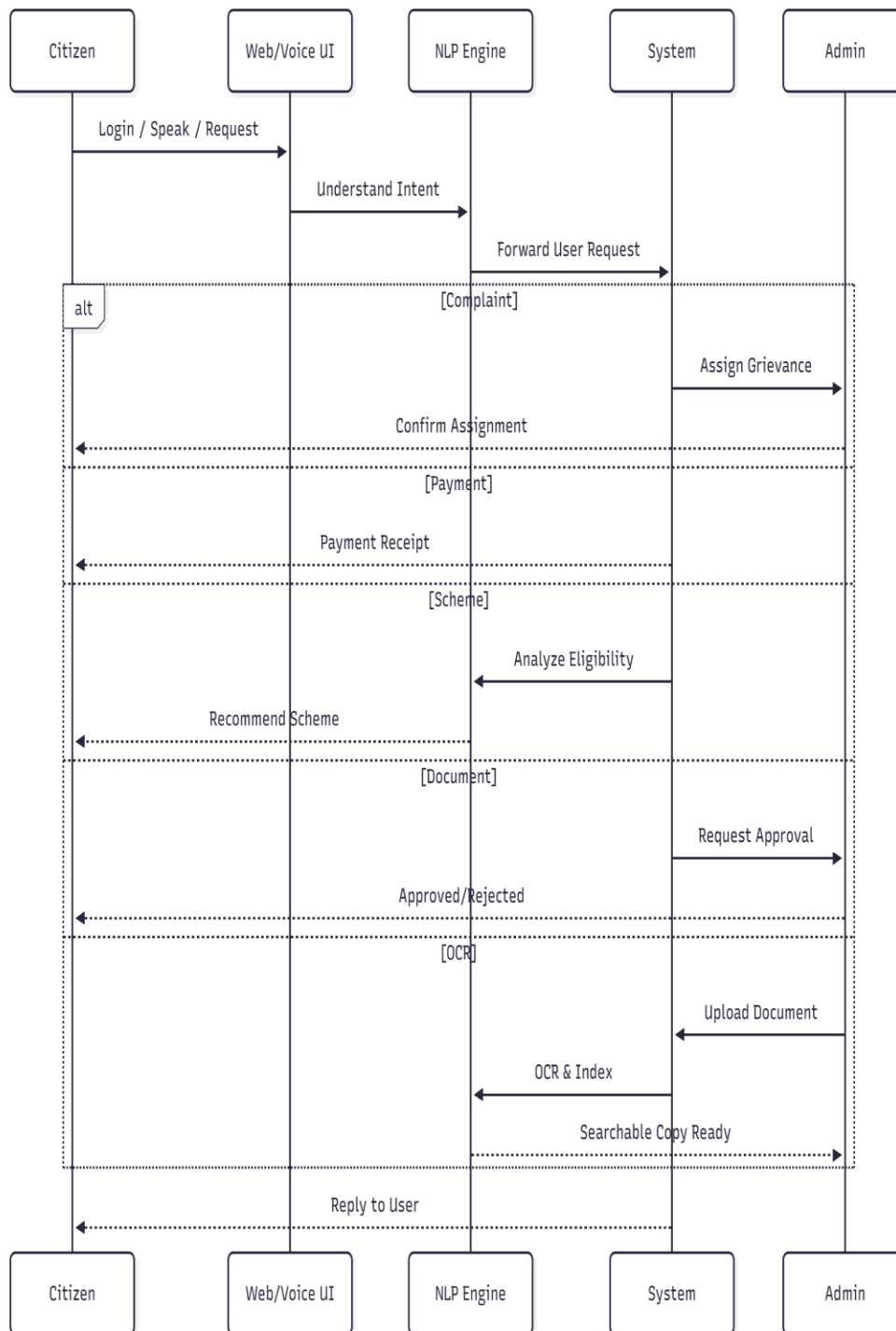


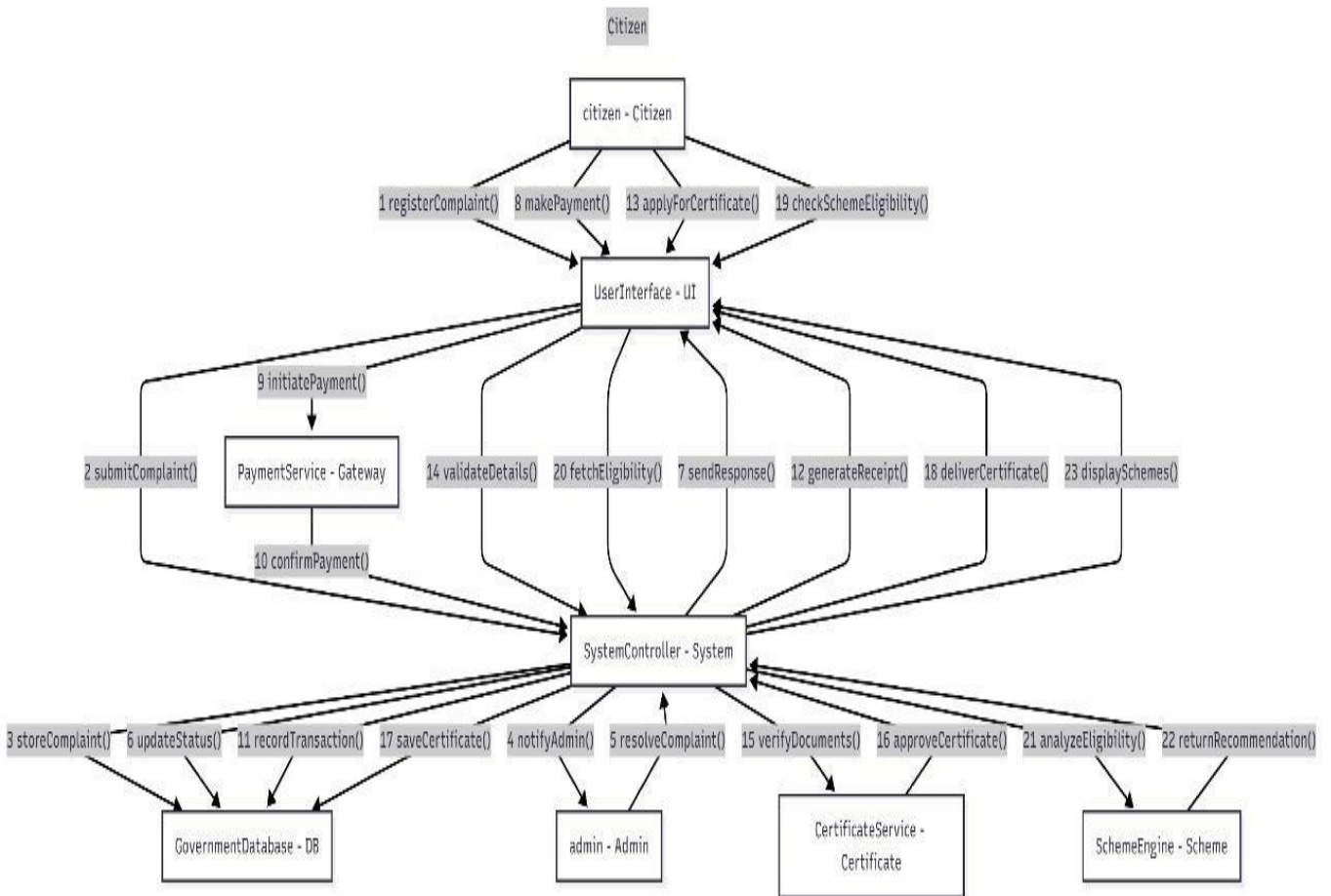
(Dfd 1)

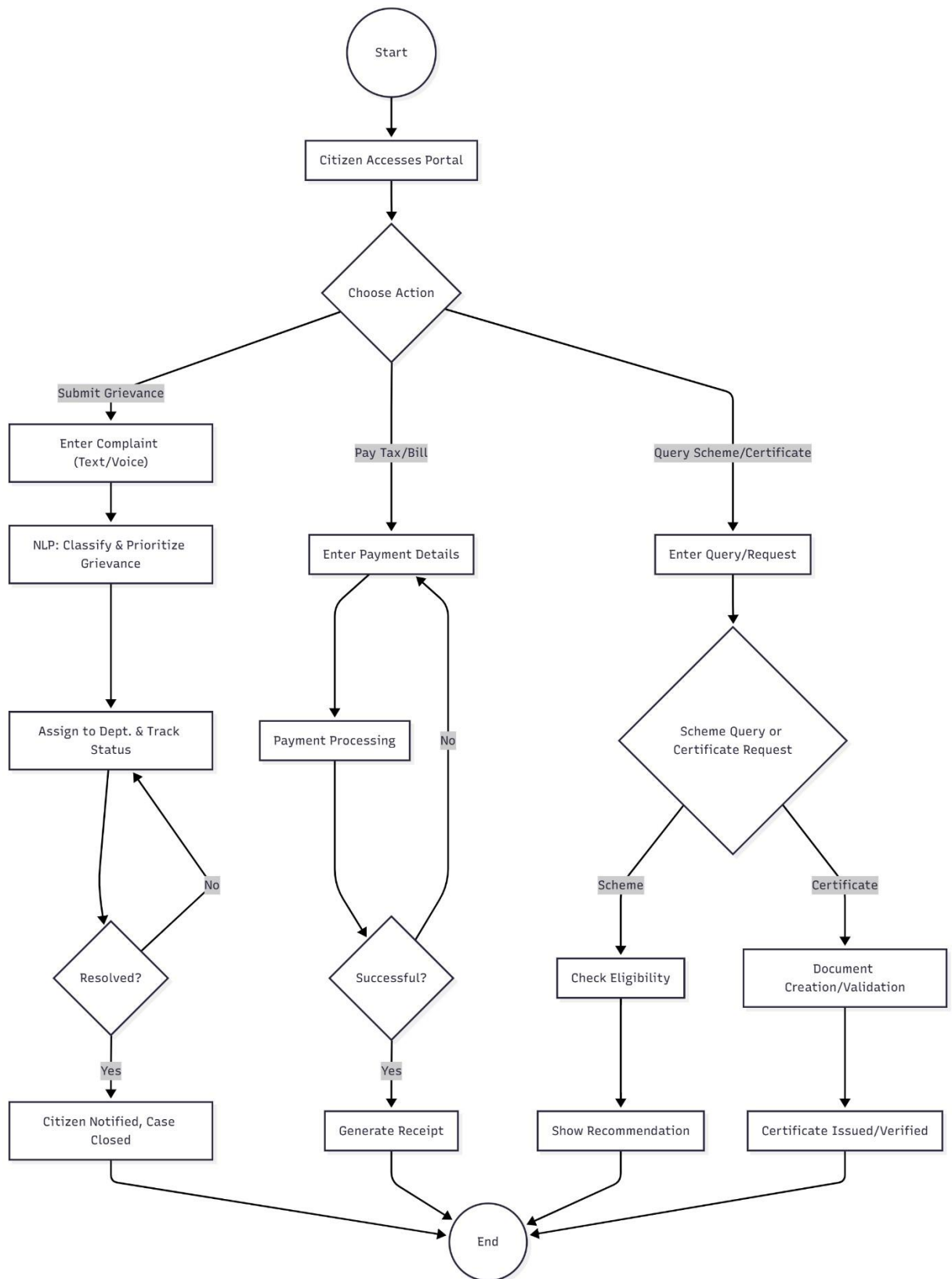


(Dfd 2)

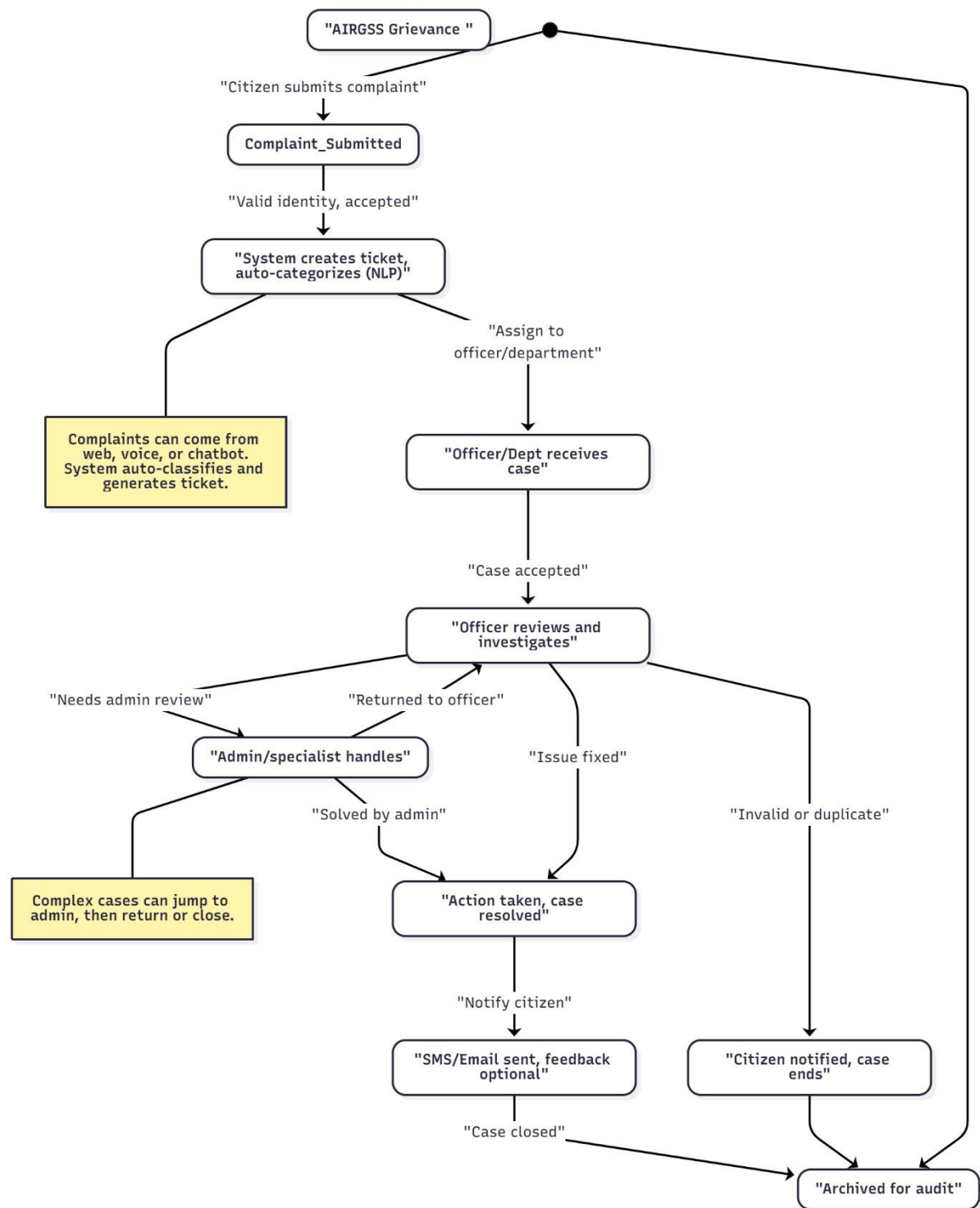
4.3. Behavioural Design

a. Sequence Diagram

b. Collaboration diagram

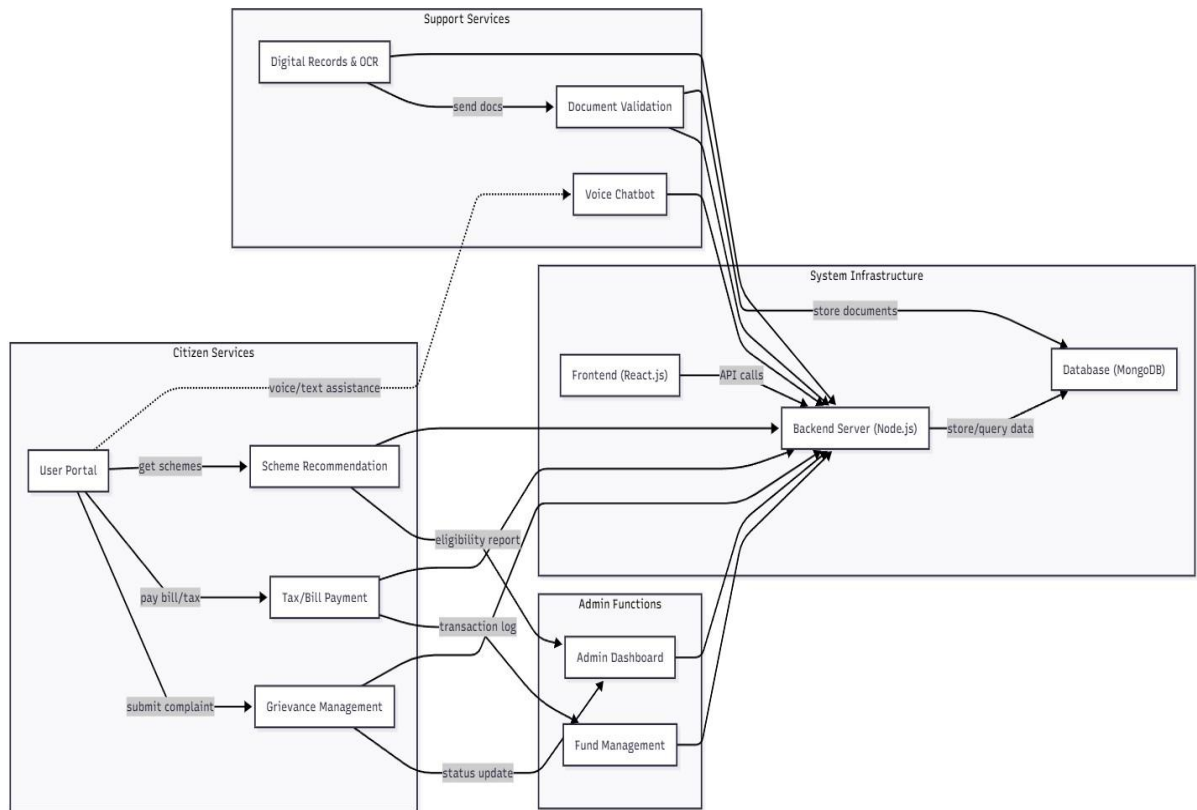
c. Activity Diagram

d. State-chart diagram



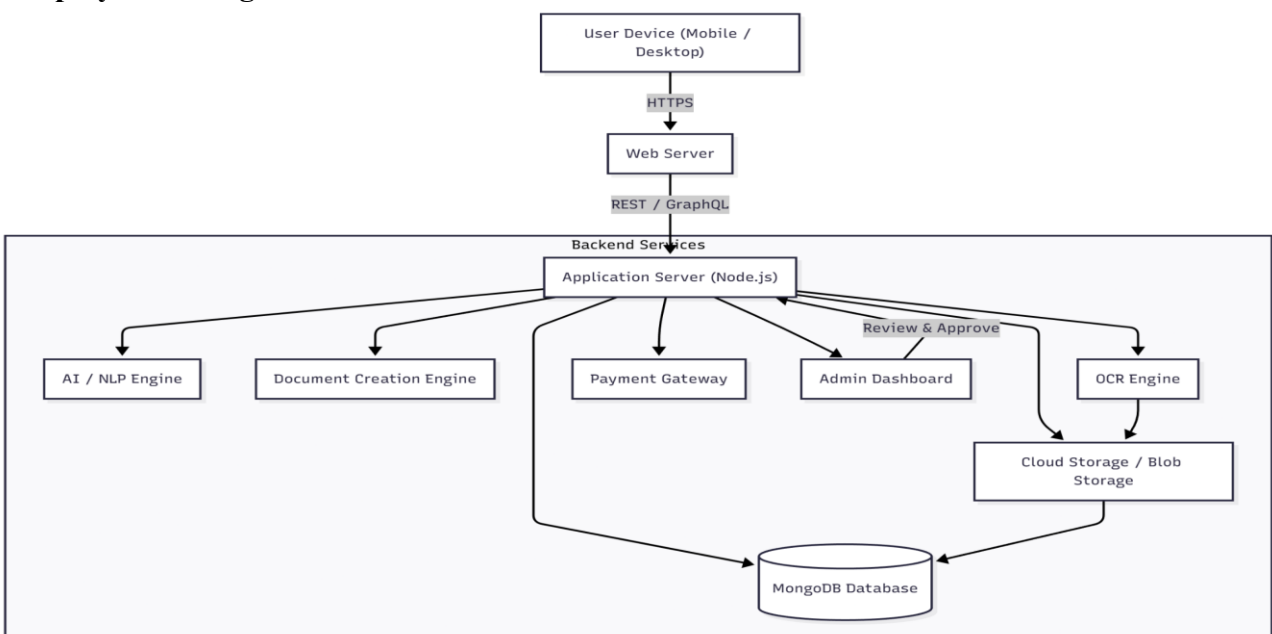
4.3. Implementation Design

a. Component Diagram



4.4. Environment Design

a. Deployment Diagram



5. Algorithms for Functional Requirements

Grievance Management

Input: A new citizen grievance submission (text or voice) and the submitter's profile/credentials.

Output: Logged grievance record with unique ID, notifications to citizen and responsible officers, and updated grievance status history.

1. Receive the grievance from the user interface (mobile app or voice channel).
2. **Voice Input Handling:** If the submission is voice-based, run speech-to-text (STT) to obtain the grievance text.
3. **NLP Analysis:** Use an NLP module to parse the text, extract keywords, and classify the grievance category (e.g., infrastructure, health, education) and priority (e.g., urgent, normal).
4. **Identity Verification:** Authenticate the citizen using stored credentials (e.g., login session or OTP) and retrieve the citizen's profile from the database. Verify identity to prevent spam.
5. Create a new grievance record in the Grievance Management database with fields: grievance ID (auto-generated), user ID, timestamp, category, priority, and description.
6. **Assignment:** Consult rules or machine-learning model to assign the grievance to an appropriate officer or department based on category, location, and workload. Update the record with assignee.
7. **Notification to Stakeholders:** Send an acknowledgment to the citizen (via SMS/email) containing the grievance ID and estimated response time. Notify the assigned officer or department (via email/SMS/internal alert) of the new case.
8. Log the initial status as "Received" and allow the officer to update progress. Each status update (In Review, In Progress, Resolved) is recorded in the grievance log with timestamp.
9. **Escalation/Administration:** If the grievance is complex or flagged (e.g., sensitive content), escalate it to an administrator for review. Admin can reassign or intervene as needed.
10. Once resolved, record the resolution details and close the loop by notifying the citizen of the outcome. Archive the case for audit.

Scheme Eligibility & Recommendation

Input: Citizen's profile data (demographics, income, assets, family status, location, etc.) and a database of government schemes with eligibility criteria.

Output: A list of schemes the user is eligible for, ranked by relevance or benefit, and guidance on how to apply.

1. **Profile Retrieval:** Fetch the user's complete profile from the user database (age, gender, income level, social category, family details, etc.).
2. Load all active government schemes from the Schemes database. Each scheme entry includes eligibility rules, benefits, required documents, and application process.
3. For each scheme, apply the eligibility rules (e.g., income thresholds, age limits, location restrictions) to the user's profile. Determine a boolean eligible/not-eligible and calculate any matching score or benefit.
4. Compile a list of all schemes for which the user meets the eligibility criteria.
5. **Ranking/Recommendation:** If multiple schemes apply, rank them by priority (e.g., highest benefit amount, urgency, or application deadline) using a simple scoring model.
6. Present the recommended schemes to the user via the interface, including summary of each scheme's purpose and the steps to apply.
7. For each recommended scheme, provide links or forms to start the application. If the application requires documents, flag it for the Document Creation & Validation module (e.g., to generate a certificate).
8. If the user asks questions (e.g., "Why am I eligible?"), use NLP to fetch relevant criteria from the scheme data and explain.
9. **Admin Verification (Optional):** For certain sensitive schemes (like large subsidies), flag the recommendation for admin review; the admin can verify eligibility before the user proceeds.
Log the recommendation session (which schemes were shown, user selections) for future audit and system improvement.

Document Creation & Validation

Input: Request details and user data for generating an official document (e.g., certificates, letters) or an uploaded document for authenticity check.

Output: Generated official document (e.g., PDF) with digital signature/QR code or a validation report confirming authenticity.

1. **Template Selection:** The user (or admin on behalf of user) selects the type of document to generate (birth certificate, permit, etc.). The system retrieves the corresponding template.
2. Gather required data fields (name, date, purpose) from the user's profile or request form. If missing, prompt user to input them.

3. Populate the template: merge data into the official layout (adding government letterhead, formatting).
4. Apply digital signing if required: use a cryptographic signature or QR code to ensure authenticity and enable easy validation. Embed a unique document ID.
5. Save the generated document in the Document Management database and provide a download link to the user. Notify the user when the document is ready.
6. **Validation (if checking a user's existing document):** If a user uploads a governmentissued document to be validated, extract the document ID or QR code. Verify the signature/ID against the database.
7. If validating, respond with a report: either a confirmation that the document is genuine (matching a record) or an error stating it could not be validated.
8. Admin Verification Step: If policies require, send the newly generated document for admin approval before delivery. The admin reviews and clicks "Approve," which finalizes the process.
9. Maintain a log of generated documents: who created them, timestamps, and status (pending approval, issued, etc.).
10. Update the user's record to link to the new document (for later access or audits).

6. References

- [1] Y. Jiang *et al.*, “Natural Language Processing Adoption in Governments and Future Research Directions: A Systematic Review,” *Applied Sciences*, vol. 13, no. 22, 2023, Art. no. 12346.
- [2] K. K. Nirala, N. K. Singh, and V. S. Purani, “A survey on providing customer and public administration based services using AI: chatbot,” *Multimedia Tools and Applications*, vol. 81, pp. 22 215–22 246, 2022.
- [3] S. Senadheera, T. Yigitcanlar, K. C. Desouza, and P. H. Cheong, “Understanding Chatbot Adoption in Local Governments: A Review and Framework,” *Journal of Urban Technology*, 2024 (online).
- [4] M. Hakimi, H. Ghafory, and A. W. Fazil, “Enterprise Architecture in E-Government: A Study of Integration Challenges and Strategic Opportunities,” *Int’l Journal of Software Engineering and Computer Science*, vol. 4, no. 2, pp. 430–452, 2024.
- [5] I. Siahaan *et al.*, “Community Welfare in Medan City Government: The Role of Accountability, Transparency, and Apparatus Performance,” *Int’l Journal of Management Science and Social Science Research*, Jan. 2025.
- [6] A. Zuidervijk, Y.-C. Chen, and F. Salem, “Implications of the Use of Artificial Intelligence in Public Governance: A Systematic Literature Review and a Research Agenda,” *Government Information Quarterly*, vol. 38, 101577, 2021.
- [7] I. Savveli, M. Rigou, and S. Balaskas, “From E-Government to AI E-Government: A Systematic Review of Citizen Attitudes,” *Informatics*, vol. 12, no. 3, 2025, Art. no. 98.
- [8] C. Potts *et al.*, “Chatbots to Support Mental Wellbeing of People Living in Rural Areas: Can User Groups Contribute to Co-design?,” *Journal of Technology in Behavioral Science*, vol. 6, no. 4, pp. 652–665, 2021.
- [9] United Nations, *E-Government Survey 2024: Digital Government in the Decade of Action for Sustainable Development*, UN DESA, 2024.